

IVFFlat 内存自适应构建与诊断增强

技术报告与复现指南

OpenTenBase / PostgreSQL 18 • pgvector 0.8.6 • Rhino-Bird 项目交付

姓名：李星谕

电话号码：17360149608

项目定位

面向 IVFFlat 构建阶段的内存瓶颈与参数可理解性问题：在保留 Elkan fast path 的前提下，加入低内存 Yinyang fallback、生命周期感知内存估算和结构化配置诊断，提升低内存场景的可构建性、失败可解释性与参数使用体验。

1 项目概述

IVFFlat 在构建索引时需要通过 K-Means 生成聚类中心。现有 Elkan 路径依靠逐样本、逐中心下界减少距离计算，但核心 lowerBound 数组按 $O(NK)$ 增长，在较大的 lists 场景下容易触及 maintenance_work_mem。

本项目在 OpenTenBase / PostgreSQL 18 + pgvector 0.8.6 中集成基于 Yinyang Global/Group Filter 的低内存 K-Means 路径，并在实际采样完成后估算两条路径的峰值内存：Elkan 可行时保持原 fast path；Elkan 超预算而 Yinyang 可行时自动回退；两者均不可行时在大数组分配前返回 ERROR / DETAIL / HINT。

同时提供 ivfflat_index_diagnostics(index regclass) 主动诊断接口，对 lists、probes、统计信息和构建资源给出结构化风险提示，不进入普通 ANN 查询热路径，也不自动修改用户参数。

2 快速复现

最小复现不依赖外部数据下载，直接在本地 PostgreSQL/OpenTenBase 实例上创建确定性小数据集，并验证自动 fallback、索引可用性和 diagnostics：

```
bash contrib/pgvector/bench/rhino/reproduce_smoke.sh
```

预期关键输出：

- automatic Yinyang fallback selected; CREATE INDEX succeeded;
含义：Elkan 已超出当前内存预算，构建器自动切换到可行的 Yinyang 路径，并且索引实际创建成功。
- planner uses demo_vectors_ivfflat_idx; query returned 5 rows;
含义：新建的 IVFFlat 索引能够被优化器实际采用，并完成预期的向量检索，而不是只“建出来”但不可用。
- lists_high / low_recall_risk / build_memory 均被结构化诊断；
含义：诊断接口能够识别 lists 偏高、probes 过低带来的召回风险，并报告构建内存相关信息。
- 最终输出 Rhino-Bird IVFFlat reproduction: PASS。
含义：最小复现中的 fallback、索引可用性和 diagnostics 断言全部通过，整条 smoke 流程完成。

复现边界

该脚本用于最小功能复现，不等同于 SIFT / GloVe / GIST 正式 benchmark。完整实验环境、原始构建时间、Recall、数据集 SHA256 与证据来源见配套文档《IVFFlat 实验环境与详细实验数据》。

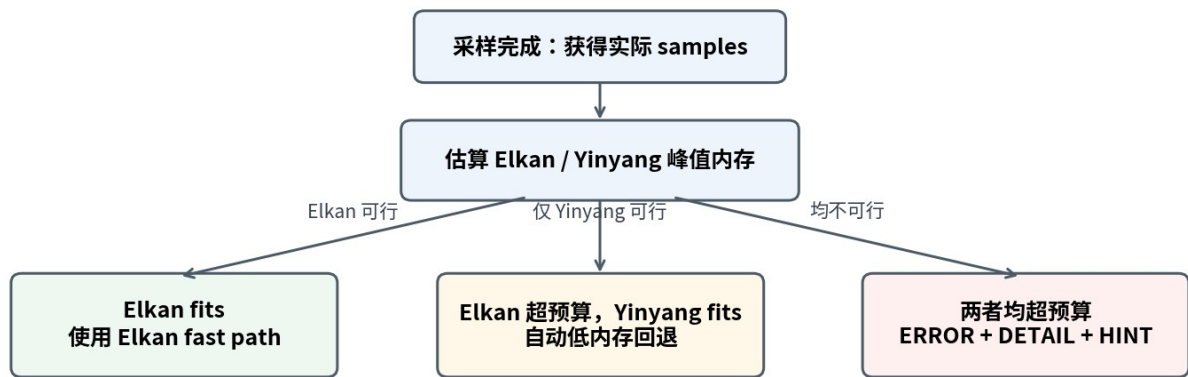
3 实施方案

3.1 内存模型与低内存路径

Elkan 的主要空间项可简化为 $M_{\text{lower}} = N \times K \times \text{sizeof}(\text{float})$ ，其中 N 为实际采样数， K 与 `lists` 一致。Yinyang 将中心划分为更少的组，并通过 Global Filter 与 Group Filter 维护更粗粒度的下界；主要分组下界约为 $M_{\text{group}} = N \times G \times \text{sizeof}(\text{float})$ ，且 $G \ll K$ 。

Yinyang 路径还采用低内存 k-means++ 初始化，仅保留 `weight[N]` 并持续更新最近中心距离，避免初始化阶段再形成 $N \times K$ 级数组。内存估算按对象实际生命周期建模：初始化数组和主循环数组不重叠时取两阶段最大值，`groupCenters`、`agg`、`pos` 等临时对象按真实重叠关系计入。

3.2 自适应选择策略



选择依据只使用 `maintenance_work_mem` 可行性；不自动修改 `lists` / `probes`

图 1 IVFFlat K-Means 内存自适应选择流程

选择发生在 `SampleRows()` 完成之后，使用实际 `samples->length` 与当前 `maintenance_work_mem`。算法选择只依据构建可行性，不自动改写 `lists` 或 `probes`。这样可以让内存充足的工作负载继续使用原 Elkan fast path，只在必要时支付低内存路径的额外计算开销。

3.3 实机 fallback 行为

在确定性 10,000 行 / `vector(3)` 最小场景中，`maintenance_work_mem=16MB`、`lists=1000` 时，Elkan 估算约 43MB，Yinyang 约 5MB。系统自动选择 Yinyang 并完成 CREATE INDEX：

```

SET
rhino_demo=#
rhino_demo=# SET maintenance_work_mem = '16MB';
SET
rhino_demo=# CREATE INDEX demo_vectors_ivfflat_idx
rhino_demo=# ON demo_vectors
rhino_demo=# USING ivfflat (v vector_l2_ops)
rhino_demo=# WITH (lists = 1000);
DEBUG: building index "demo_vectors_ivfflat_idx" on table "demo_vectors" serially
DEBUG: using Yinyang k-means for IVFFlat build: Elkan exceeds maintenance_work_mem and Yinyang fits (Elkan=43661 kB/43 MB, Yinyang=4569 kB/5 MB, maintenance_work_mem=16384 kB/16 MB)
CREATE INDEX
  
```

图 2 Elkan 超预算、Yinyang 可行时自动回退并完成索引构建

4 配置诊断能力

ivfflat_index_diagnostics(index regclass) 是只读、显式调用的诊断接口。它读取 estimated rows、lists、当前 probes 以及构建相关设置，输出结构化 issue / severity / current value / recommendation。诊断逻辑避免为了“精确”而扫描全表。

- lists: 以 $\text{rows} \leq 1\text{M}$ 时 $\text{ceil}(\text{rows}/1000)$ 、更大规模时 $\text{ceil}(\text{sqrt}(\text{rows}))$ 为起始参考；只有明显偏离时才提示。
- probes: 以 $\text{ceil}(\text{sqrt}(\text{lists}))$ 为起始参考；明显偏低时提示 low_recall_risk，接近 lists 时提示查询代价。
- 统计信息: 缺失或过旧时给出 ANALYZE 建议，不执行额外全表计数。
- build_memory: 作为构建资源信息展示；不把 maintenance_work_mem 与进程 RSS 混为一谈。

诊断接口只提示“低召回风险”，不在没有 ground truth 的情况下预测具体 Recall 数值；NULL 向量和 cosine 零向量等需要扫描数据才能准确计数的情况也不做伪精确判断。

```
border style ts 2;
rhino_demo=# SELECT
rhino_demo-#     severity,
rhino_demo-#     issue,
rhino_demo-#     current_value,
rhino_demo-#     recommended_value
rhino_demo-# FROM ivfflat_index_diagnostics('demo_vectors_ivfflat_idx':regclass)
rhino_demo-# WHERE issue IN ('lists_high', 'low_recall_risk', 'build_memory');
```

severity	issue	current_value	recommended_value
WARNING WARNING INFO	lists_high low_recall_risk build_memory	lists=1000, estimated_rows=10000, rows_per_list=10.00 probes=1, lists=1000 maintenance_work_mem=16384 kB, lists=1000	recommended starting point about 10 lists recommended probes starting point about 32 increase maintenance_work_mem if build memory checks fail

(3 rows)

图 3 lists=1000、probes=1 时的 IVFFlat 参数与低召回风险诊断

5 复现路径

评审可以按“最小复现 → 扩展失败边界 → 正式实验数据”三层检查：

① 最小复现	reproduce_smoke.sh: 自动 fallback、CREATE INDEX、Index Scan、diagnostics。
② 扩展边界	reproduce_smoke.sh --extended: 验证两条算法均不满足预算时的 ERROR / DETAIL / HINT。
③ 正式实验	配套《实验环境与详细实验数据》：SIFT / GloVe / GIST、原始 timing、Recall、SHA256、测试证据。

6 核心实验结果

6.1 实验环境与口径

正式验证环境为 OpenCloudOS 8.10 x86_64, 4 vCPU (AMD Ryzen 7 8845HS), 11 GiB RAM; 数据库为 PostgreSQL 18.6 / OpenTenBase REL_18_STABLE 基线, pgvector 0.8.6。正式 workload 覆盖 SIFT-128 (L2)、GloVe-100 (cosine/angular) 和 GIST-960 (L2)。

下文“内存”均指 IVFFlat K-Means estimator / maintenance_work_mem 可行性估算值, 不是进程 RSS、peak RSS 或物理内存。查询质量采用 Recall@10, 1000 queries。

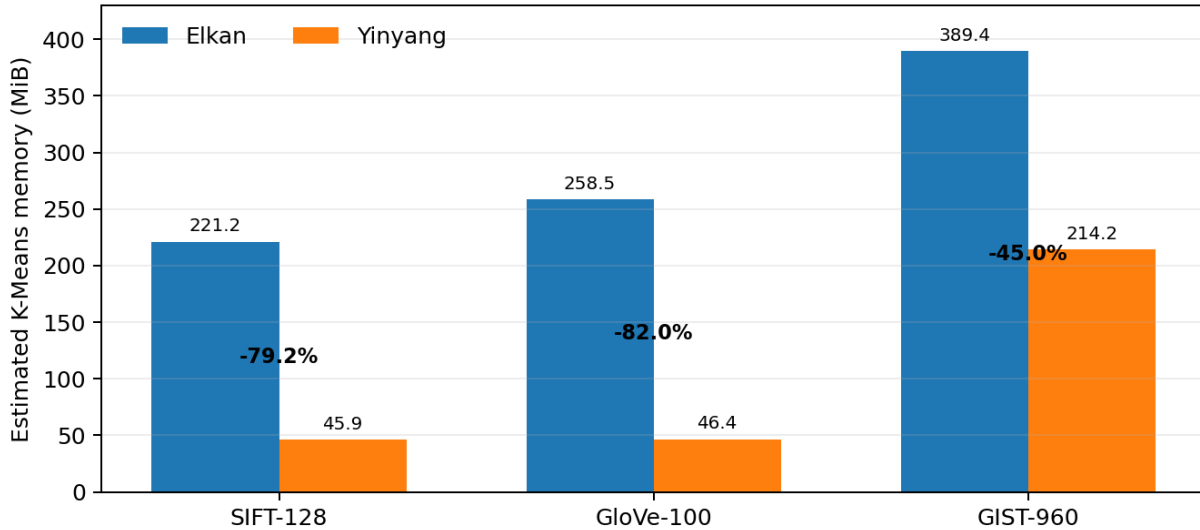


图 4 三类 workload 下 Elkan 与 Yinyang 的 K-Means 内存估算

Workload	Elkan MiB	Yinyang MiB	下降	低内存结果	Recall@10
SIFT-128	221.218	45.939	79.23%	128MB: E 失败 / Y 成功	0.9769 / 0.9762
GloVe-100	258.513	46.404	82.05%	256MB: E 失败 / Y 成功	0.8744 / 0.8746
GIST-960	389.431	214.152	45.01%	256MB: E 失败 / Y 成功	0.9313 / 0.9356

表 1 已核验的内存估算、低预算可构建性与 Recall@10

6.2 构建时间解释

Yinyang 的稳定价值是降低构建阶段的 K-Means 内存估算并扩大低预算下的可构建范围, 而不是保证所有 workload 更快。SIFT D3 的 10-pair 整体 CREATE INDEX 配对中位比显示 Yinyang 约快 11.58%; GloVe 与 GIST 的三次公平运行中位数则分别慢 27.77% 和 26.98%。

SIFT fixed-seed phase diagnostic 还显示, Yinyang 的 K-Means 阶段本身并不更快。因此本项目不把总体构建收益表述为“K-Means 普遍加速”, 而采用 Elkan fast path + Yinyang low-memory fallback。

7 验证状态

- pgvector regression / installcheck: 14 / 14 PASS;
- adaptive K-Means TAP: 20 / 20 PASS;
- IVFFlat diagnostics TAP: 29 / 29 PASS;
- integration smoke: 覆盖 Elkan fast path、Yinyang fallback、both-fail、fallback 后索引可用性;
- 最小复现脚本: 默认运行、KEEP_DB、重复运行与 --extended 均已验证。

8 复现范围与限制

- Yinyang / group-bound K-Means 是已有公开算法思想; 本项目贡献是 pgvector/OpenTenBase 中的低内存实现、生命周期内存建模、自适应选择和工程诊断。
- 当前范围面向 PostgreSQL 18 / OpenTenBase centralized 环境的 IVFFlat; 不覆盖 HNSW, 也不声明分布式构建支持。
- 内存结果是 K-Means estimator / maintenance_work_mem budget estimate, 不等同于进程 RSS; 没有据此声称物理内存按同等比例下降。
- 构建时间具有 workload 依赖性; 不声称 Yinyang 在所有数据集上更快。
- Recall@10 基本保持, 但不声称与 Elkan 完全一致。
- 未提供 CPU cache miss、内存带宽等 hardware-counter 结论, 也不据此解释性能差异。

9 交付与代码位置

最终 OpenTenBase fork / branch:

<https://github.com/iamkuangzhang/OpenTenBase/tree/rhino-ivfflat-adaptive-kmeans>

最小复现脚本: contrib/pgvector/bench/rhino/reproduce_smoke.sh

配套数据文档: 《IVFFlat 实验环境与详细实验数据》

10 参考与依据

- [1] pgvector: IVFFlat documentation and parameter guidance.
- [2] Elkan, C. Using the Triangle Inequality to Accelerate k-Means. ICML 2003.
- [3] Ding, Y. et al. Yinyang K-Means. ICML 2015.
- [4] Newling, J.; Fleuret, F. Fast k-means with accurate bounds. ICML 2016.
- [5] PostgreSQL 18 Documentation: maintenance_work_mem.

核心方案总结

原有 IVFFlat 构建采用 Elkan K-Means, 在较大的 lists / 样本规模下, 其 $O(NK)$ 下界数组可能使 K-Means 内存估算超过 maintenance_work_mem。我们参考公开的 Yinyang K-Means, 引入 Global / Group Filter 的低内存路径。三组正式 workload 中, K-Means 内存估算分别下降 79.23%、82.05% 和 45.01%, Recall@10 基本保持; 构建时间则依 workload 而异, SIFT 快 11.58%, GloVe / GIST 慢约 27%。因此最终方案不是用 Yinyang 全面替换 Elkan, 而是 Elkan 可行时继续走原 fast path, 仅在 Elkan 超预算且 Yinyang 可行时自动切换; 用户无需手动选择算法, 只有两条路径都不满足预算时才收到明确的 ERROR / DETAIL / HINT。同时, 项目新增 lists / probes / 统计信息 / build_memory 的结构化诊断, 进一步改善 IVFFlat 的构建与参数使用体验。